

CustomElevatedButton

A highly customizable Flutter button widget that supports:

-  Solid color or gradient background
-  Left SVG icon and/or right Material icon
-  Custom size, padding, border, and elevation
-  Reusable design system friendly API
-  Ink ripple support with gradient backgrounds

This widget is ideal for building consistent buttons across your Flutter app while keeping flexibility for different UI needs.

Imports

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';
import 'package:isaacmwesigw352/assets_helper/app_colors.dart';
import 'package:isaacmwesigw352/assets_helper/app_fonts.dart';
```

Constructor Parameters

Required

Parameter	Type	Description
<code>text</code>	<code>String</code>	Button label text

Callbacks

Parameter	Type	Description
<code>onTap</code>	<code>VoidCallback?</code>	Called when button is tapped

Layout & Styling

Parameter	Type	Description
<code>width</code>	<code>double?</code>	Button width (default: full width)

Parameter	Type	Description
<code>height</code>	<code>double?</code>	Button height (default: 52)
<code>backgroundColor</code>	<code>Color?</code>	Solid background color
<code>textColor</code>	<code>Color?</code>	Text color
<code>borderRadius</code>	<code>double?</code>	Corner radius
<code>elevation</code>	<code>double?</code>	Button elevation
<code>padding</code>	<code>EdgeInsetsGeometry?</code>	Inner padding
<code>textStyle</code>	<code>TextStyle?</code>	Custom text style

Icon Support

Left SVG Icon

Parameter	Type	Description
<code>leftIconAsset</code>	<code>String?</code>	SVG asset path
<code>leftIconSize</code>	<code>double?</code>	Icon size
<code>leftIconColor</code>	<code>Color?</code>	Icon color
<code>leftIconSpacing</code>	<code>double?</code>	Space after icon
<code>leftIconPadding</code>	<code>EdgeInsetsGeometry?</code>	Custom padding around icon

Right Material Icon

Parameter	Type	Description
<code>rightIcon</code>	<code>IconData?</code>	IconData for right icon
<code>iconSize</code>	<code>double?</code>	Icon size
<code>iconColor</code>	<code>Color?</code>	Icon color
<code>iconSpacing</code>	<code>double?</code>	Space before icon

Border Options

Parameter	Type	Description
<code>borderColor</code>	<code>Color?</code>	Border color
<code>borderWidth</code>	<code>double?</code>	Border width

Gradient Support

Parameter	Type	Description
<code>useGradient</code>	<code>bool</code>	Enables gradient background
<code>gradient</code>	<code>Gradient?</code>	Custom gradient

When `useGradient` is `true`, the button uses a `Container + InkWell` instead of `ElevatedButton` to support gradients with ripple effects.

Usage Examples

Simple Button

```
CustomElevatedButton(  
  text: 'Continue',  
  onTap: () {},  
)
```

Button with Left SVG Icon

```
CustomElevatedButton(  
  text: 'Login with Google',  
  leftIconAsset: 'assets/icons/google.svg',  
  onTap: () {},  
)
```

Gradient Button with Border

```
CustomElevatedButton(  
  text: 'Get Started',  
  useGradient: true,
```

```
borderRadius: 16,  
borderColor: Colors.white,  
borderWidth: 1.5,  
onTap: () {},  
)
```

Behavior Notes

- Automatically centers content
- Icons are optional and conditionally rendered
- Full width by default
- Uses `InkWell` to preserve ripple on gradients
- Safe defaults for spacing and sizing

Full Source Code

```
import 'package:flutter/material.dart';  
import 'package:flutter_svg/flutter_svg.dart';  
import 'package:isaacmwesigw352/assets_helper/app_colors.dart';  
import 'package:isaacmwesigw352/assets_helper/app_fonts.dart';  
  
class CustomElevatedButton extends StatelessWidget {  
  final String text;  
  final VoidCallback? onTap;  
  
  final double? width;  
  final double? height;  
  final Color? backgroundColor;  
  final Color? textColor;  
  final double? borderRadius;  
  final double? elevation;  
  final TextStyle? textStyle;  
  final EdgeInsetsGeometry? padding;  
  
  final IconData? rightIcon;  
  final double? iconSize;  
  final Color? iconColor;  
  final double? iconSpacing;  
  
  final String? leftIconAsset;  
  final double? leftIconSize;  
  final Color? leftIconColor;  
  final double? leftIconSpacing;
```

```

final EdgeInsetsGeometry? leftIconPadding;

final Color? borderColor;
final double? borderWidth;

final bool useGradient;
final Gradient? gradient;

const CustomElevatedButton({
  super.key,
  required this.text,
  this.onTap,
  this.width,
  this.height,
  this.backgroundColor,
  this.textColor,
  this.borderRadius,
  this.elevation,
  this.textStyle,
  this.padding,
  this.rightIcon,
  this.iconSize,
  this.iconColor,
  this.iconSpacing,
  this.leftIconAsset,
  this.leftIconSize,
  this.leftIconColor,
  this.leftIconSpacing,
  this.leftIconPadding,
  this.borderColor,
  this.borderWidth,
  this.useGradient = false,
  this.gradient,
});

@override
Widget build(BuildContext context) {
  final bool hasRightIcon = rightIcon != null;
  final bool hasLeftIcon = leftIconAsset != null;

  final buttonChild = Row(
    mainAxisAlignment: MainAxisAlignment.min,
    mainAxisSize: MainAxisSize.min,
    children: [
      if (hasLeftIcon)
        Padding(
          padding: leftIconPadding ??
            EdgeInsets.only(right: leftIconSpacing ?? 8),

```

```

        child: SvgPicture.asset(
          leftIconAsset!,
          height: leftIconSize ?? 24,
          width: leftIconSize ?? 24,
        ),
      ),
    Text(
      text,
      style: textStyle ??
        TextStyle(
          fontFamily: 'TextFontStyle.textStyle16w500c212B36publicSans',
          fontWeight: FontWeight.bold,
          color: textColor ?? AppColors.white,
        ),
    ),
    if (hasRightIcon) ...[
      SizedBox(
        width: iconSpacing ?? 8,
        child: Icon(
          rightIcon,
          size: iconSize ?? 20,
          color: iconColor ?? textColor ?? Colors.white,
        ),
      ),
    ],
  ],
);

return SizedBox(
  width: width ?? double.infinity,
  height: height ?? 52,
  child: useGradient
    ? Container(
        decoration: BoxDecoration(
          gradient: gradient ??
            LinearGradient(
              begin: Alignment.topCenter,
              end: Alignment.bottomCenter,
              colors: [AppColors.c4F46E5, AppColors.c3B82F6],
            ),
        ),
        borderRadius: BorderRadius.circular(borderRadius ?? 12),
        border: Border.all(
          color: borderColor ?? Colors.transparent,
          width: borderWidth ?? 1,
        ),
      ),
    child: Material(
      color: Colors.transparent,
      child: InkWell(
        borderRadius: BorderRadius.circular(borderRadius ?? 12),
        onTap: onTap,

```

```

        child: Center(child: buttonChild),
      ),
    ),
  )
: ElevatedButton(
  onPressed: onTap,
  style: ElevatedButton.styleFrom(
    backgroundColor: backgroundColor ?? AppColors.buttonColor,
    elevation: elevation ?? 2,
    padding: padding ?? EdgeInsets.symmetric(horizontal: 16),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(borderRadius ?? 12),
      side: BorderSide(
        color: borderColor ?? Colors.transparent,
        width: borderWidth ?? 1,
      ),
    ),
  ),
  child: buttonChild,
),
);
}
}

```

Summary

`CustomElevatedButton` is a production-ready, scalable button widget designed for modern Flutter applications with design system consistency and customization in mind.